

Search :

- Events & Listeners
- Logging
- Laravel Telescope
-

• يُعد إطار العمل **Laravel** واحدًا من أشهر أطر تطوير تطبيقات الويب باستخدام لغة **PHP**، وذلك لما يوفره من أدوات وتقنيات تساعد المطورين على كتابة كود منظم وسهل الصيانة. ومن أهم هذه الأدوات **Events & Listeners** و **Logging** و **Laravel Telescope**، حيث تُستخدم لتنظيم سير العمل داخل التطبيق، وتتبع الأحداث، وتسجيل الأخطاء، ومراقبة أداء التطبيق أثناء عملية التطوير. ويساعد استخدام هذه الأدوات في بناء تطبيقات أكثر احترافية وسهولة في الصيانة والتطوير.

- **Events & Listeners**

هي آلية في **Laravel** تسمح بفصل المنطق (**Business Logic**) عن الحدث نفسه.

ال-Event هو حدث يقع داخل التطبيق ويعبر عن عملية معينة حدثت بالفعل، مثل تسجيل مستخدم جديد أو إنشاء طلب شراء أو تغيير كلمة المرور. ويهدف استخدام الأحداث إلى فصل المنطق البرمجي عن الإجراءات التي تحدث بعد تنفيذ العملية، مما يجعل الكود أكثر تنظيمًا ومرونة.

فعلى سبيل المثال، عند تسجيل مستخدم جديد يمكن إنشاء حدث يسمى **User Registered**، ثم يتم إطلاق هذا الحدث بعد نجاح عملية التسجيل.

• **Event**: يُمثل حدث حصل داخل التطبيق، مثل:

• **User Registered**

• **Order Placed**

• **Password Reset**

Listener: كلاس يستمع للحدث وينفذ إجراء معين عند وقوعه، فعند إطلاق حدث **UserRegistered** يمكن أن توجد عدة **Listeners** تقوم بالمهام التالية:

- إرسال Welcome Email.
- كتابة Log.
- إرسال Notification.

الفائدة:

- تنظيم الكود.
- سهولة الصيانة.
- إمكانية ربط أكثر من Listener بنفس Event.
- فصل منطق التطبيق عن الكود الموجود داخل الـ Controllers.
- جعل الكود أكثر تنظيماً وأسهل في القراءة.
- تسهيل صيانة المشروع وإضافة ميزات جديدة.
- إمكانية إعادة استخدام الكود في أكثر من مكان.
- تحسين قابلية اختبار التطبيق.
- دعم بناء تطبيقات كبيرة وقابلة للتوسع.

تمر العملية بالمراحل التالية:

1. يحدث إجراء معين داخل التطبيق (مثل تسجيل مستخدم).
2. يتم إطلاق Event يمثل هذا الحدث.
3. يبحث Laravel عن جميع الـ Listeners المرتبطة بهذا الحدث.
4. يقوم كل Listener بتنفيذ المهمة الخاصة به.
5. يستمر التطبيق في العمل بصورة طبيعية.

ويُعرف هذا الأسلوب باسم **Observer Design Pattern**، وهو أحد أشهر أنماط تصميم البرمجيات.

• Logging

الـ **Logging** هو عملية تسجيل الأحداث والأخطاء والعمليات المهمة التي تحدث داخل التطبيق في ملفات خاصة، وذلك لمساعدة المطور على متابعة أداء التطبيق واكتشاف المشكلات بسهولة.

يعتمد **Laravel** على مكتبة **Monolog** لإدارة عملية تسجيل السجلات، وهي من أشهر المكتبات المستخدمة لهذا الغرض.

مستويات تسجيل السجلات

يوفر **Laravel** عدة مستويات لتسجيل الرسائل، منها:

- Emergency
- Alert
- Critical
- Error
- Warning
- Notice
- Info
- Debug

ويتم اختيار المستوى المناسب حسب أهمية الرسالة أو نوع الخطأ.

أهمية Logging

تكمن أهمية **Logging** في أنه يساعد المطور على:

- اكتشاف الأخطاء بسرعة.
- متابعة ما يحدث داخل التطبيق.
- تسجيل العمليات المهمة.
- تحليل أداء التطبيق.
- تسهيل عملية **Debugging**.
- معرفة أسباب الأعطال في بيئة الإنتاج (Production).

مكان حفظ ملفات السجلات

يقوم **Laravel** افتراضياً بحفظ السجلات داخل المسار:

`storage/logs/laravel.log`

كما يمكن للمطور تغيير طريقة حفظ السجلات لتكون يومية أو إرسالها إلى خدمات خارجية مثل Slack أو Syslog حسب احتياجات المشروع.

Laravel Telescope :

Laravel Telescope هو أداة رسمية مقدمة من فريق **Laravel** تُستخدم لمراقبة التطبيق أثناء التطوير، حيث توفر لوحة تحكم تعرض جميع العمليات التي تحدث داخل التطبيق بشكل مباشر، مما يسهل عملية اكتشاف الأخطاء وتحليل الأداء.

وتُعتبر Telescope من أهم أدوات الـ Debugging في Laravel.

ما الذي يمكن مراقبته باستخدام Telescope؟

تسمح Laravel Telescope بمراقبة العديد من مكونات التطبيق، مثل:

- طلبات HTTP.
- الاستعلامات الخاصة بقاعدة البيانات (Database Queries).
- الأخطاء والاستثناءات (Exceptions).
- الرسائل البريدية (Mail).
- الإشعارات (Notifications).
- الأحداث (Events).
- الـ Jobs والطوابير (Queues).
- السجلات (Logs).
- عمليات التخزين المؤقت (Cache).
- المهام المجدولة (Scheduled Tasks).

وبذلك يستطيع المطور متابعة جميع تفاصيل التطبيق من مكان واحد.

فوائد Laravel Telescope

من أهم مميزات Laravel Telescope:

- تسهيل عملية Debugging.
- اكتشاف الأخطاء بسرعة.
- مراقبة أداء قاعدة البيانات.
- متابعة الرسائل البريدية المرسلة.
- عرض الأحداث والـ Listeners المنفذة.
- تحليل طلبات المستخدمين.
- تحسين أداء التطبيق أثناء التطوير.
-

العلاقة بين Events و Logging و Telescope

تعمل هذه الأدوات معًا لتوفير بيئة تطوير قوية ومنظمة.

فمثلاً عند تسجيل مستخدم جديد:

1. يتم إطلاق حدث **User Register**.
2. يقوم **Listener** بإرسال رسالة ترحيب إلى البريد الإلكتروني.
3. يقوم **Listener** آخر بتسجيل العملية داخل ملف الـ **Logs**.
4. تعرض **Laravel Telescope** جميع هذه العمليات، بما في ذلك الطلب المرسل، والحدث، والـ **Listener**، والبريد الإلكتروني، ورسائل الـ **Logs**.

وبذلك يستطيع المطور التأكد من تنفيذ جميع الخطوات بسهولة